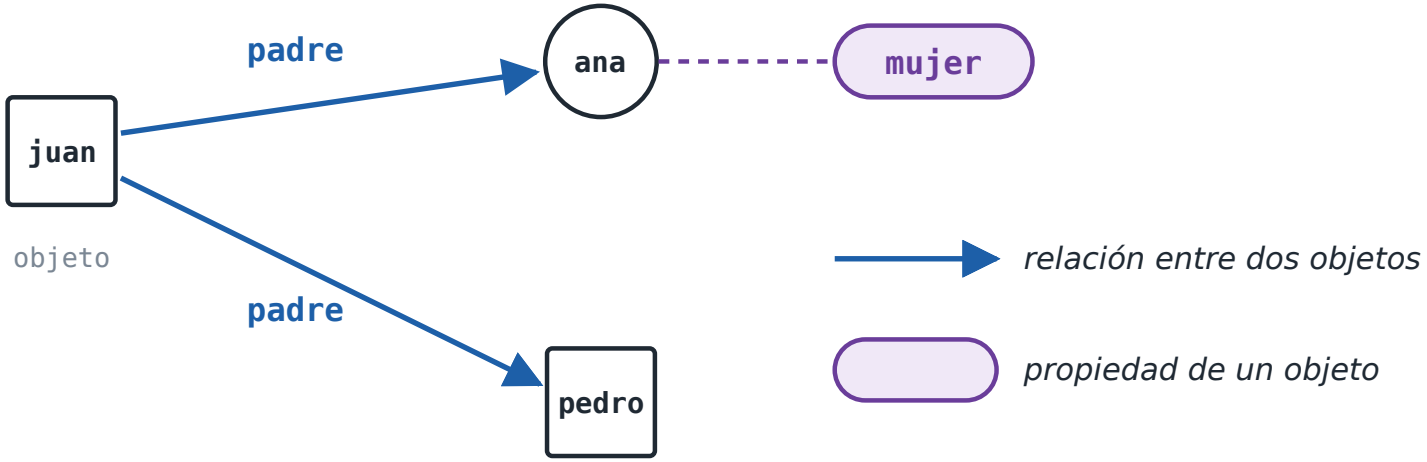
Capítulo 2 — Hechos, consultas y variables

Objetos y relaciones



Dos decisiones por relación

¿Cuántos objetos participan?

`mujer( )`	*1 objeto*	`mujer/1`
`padre( , )`	*2 objetos*	`padre/2`
`regala( , , )`	*3 objetos*	`regala/3`

quién *qué* *a quién*

¿En qué orden se escriben?

`padre(juan, ana).`

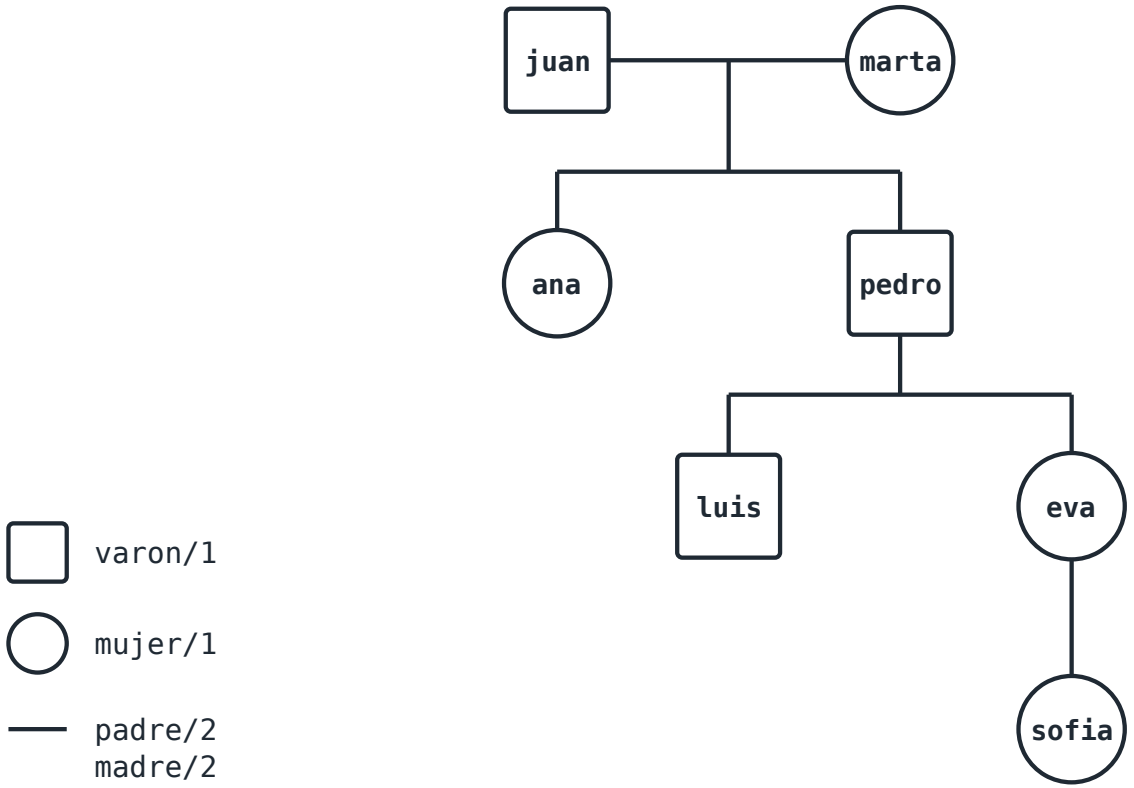
→ **juan es el padre de ana** ✓

→ ~~*ana es el padre de juan*~~

```
% padre(P, H): P es el padre de H.
```

se elige, se documenta, se respeta

La familia del capítulo

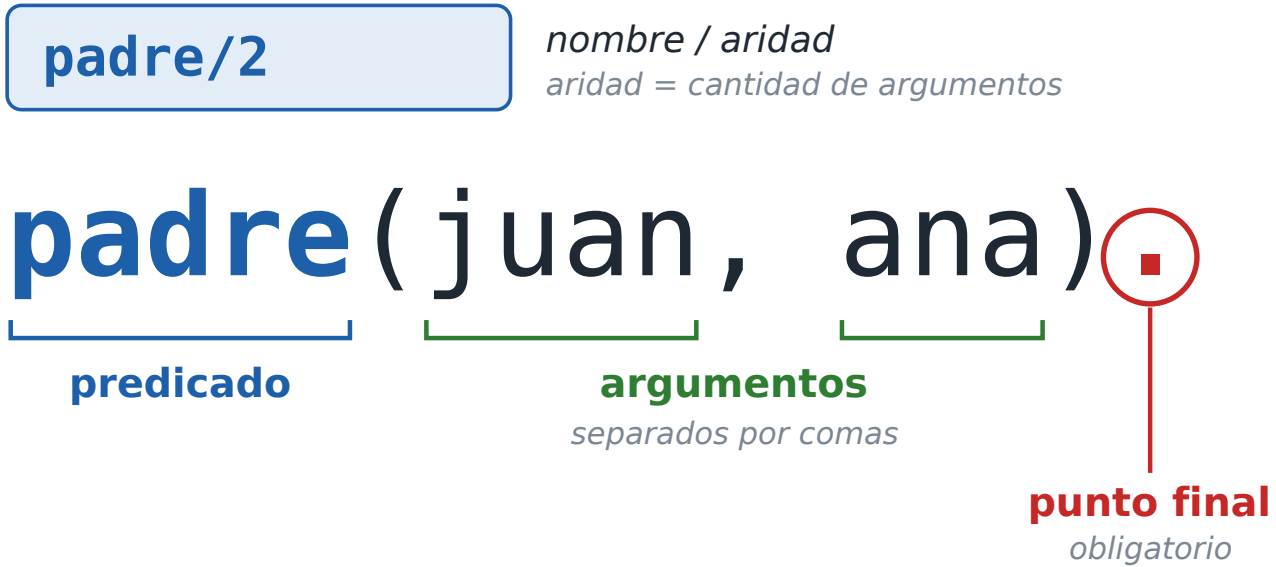


Los hechos del programa

```
% padre(P, H): P es el padre de H.
padre(juan, ana).
padre(juan, pedro).
padre(pedro, luis).
padre(pedro, eva).

% madre(M, H): M es la madre de H.
madre(marta, ana).
madre(marta, pedro).
madre(eva, sofia).
```

Anatomía de un hecho



Tres errores de sintaxis

```
2 | padre(juan, ana)       <- sin punto
ERROR: familia.pl:2:17: Syntax error: Operator expected

2 | padre(juan, ana),      <- coma en lugar de punto
ERROR: familia.pl:2:
ERROR:    Full stop in clause-body?  Cannot redefine ,/2

2 | padre (juan, ana).     <- espacio antes del paréntesis
ERROR: familia.pl:2:6: Syntax error: Operator expected
```

Átomos, números y variables

átomos

juan

prolog

'Ana'

'mi amigo'

minúscula, o entre comillas simples

números

68

3.5

sin comillas; tienen valor

variables

Ana

Quien

X

_

mayúscula o guion bajo

'Ana' con comillas es un átomo; Ana sin comillas, una variable

El nombre y la aridad

```
padre(juan, ana).
padre(juan, pedro).
```

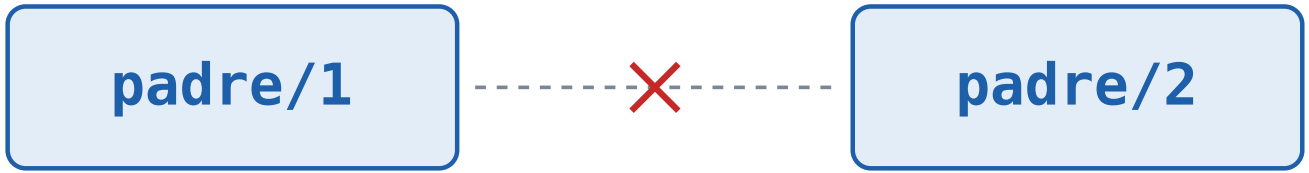
```
?- padre(juan, ana).
true.
```

=

```
hgx(juan, ana).
hgx(juan, pedro).
```

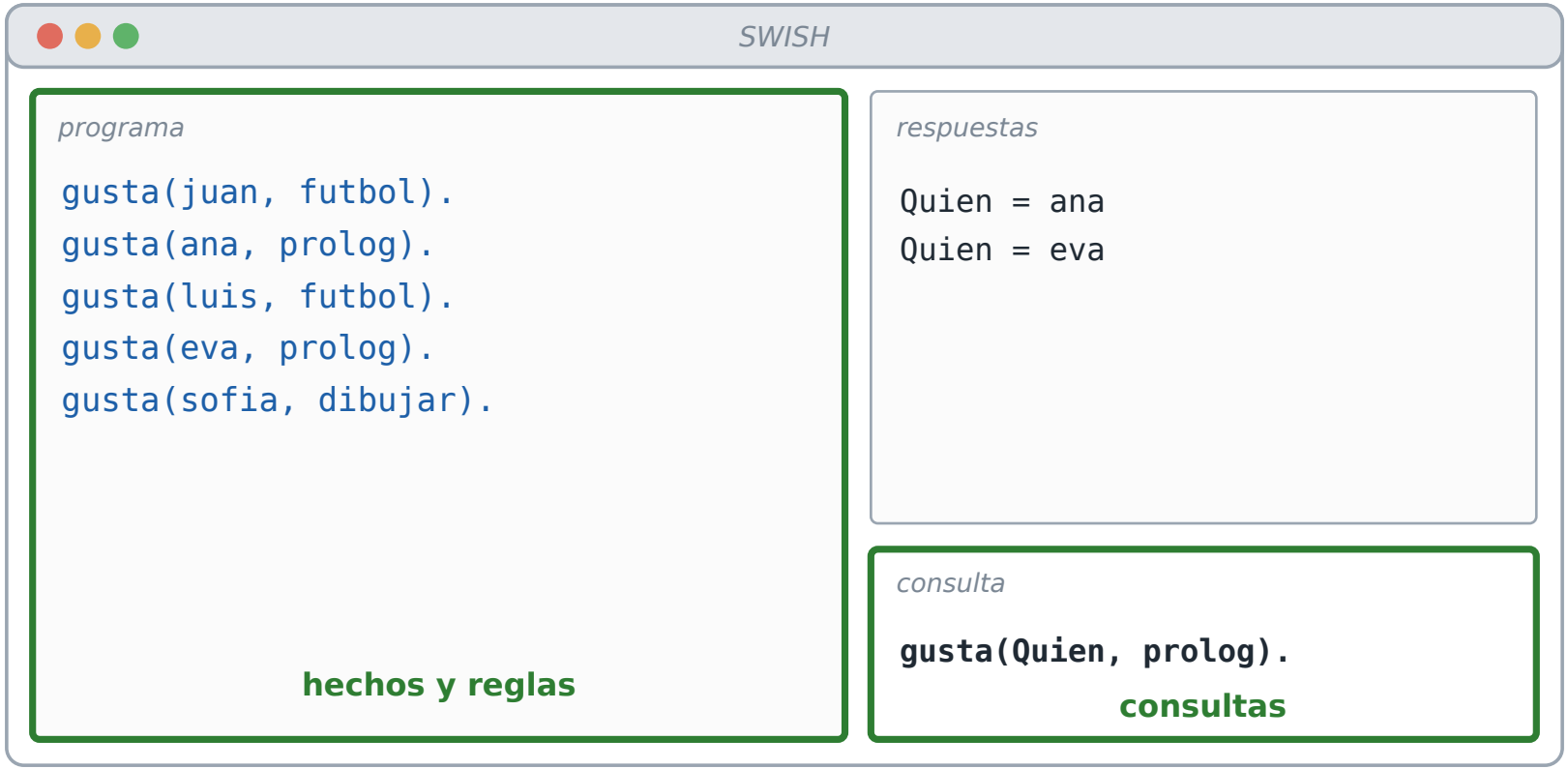
```
?- hgx(juan, ana).
true.
```

las mismas respuestas: el nombre es para quien lee



dos predicados distintos, sin relación entre sí

Dónde se escribe cada cosa: SWISH



En una instalación local

```
swipl familia.pl
```

```
?- consult('familia.pl').
true.
```

```
:- encoding(utf8).
```

Un hecho en la casilla de consultas



Una consulta sin punto final

```
?- padre(juan, ana)
|: .
true.
```

el intérprete espera el resto; el punto la completa

sin punto: la consulta sigue abierta

Consultas

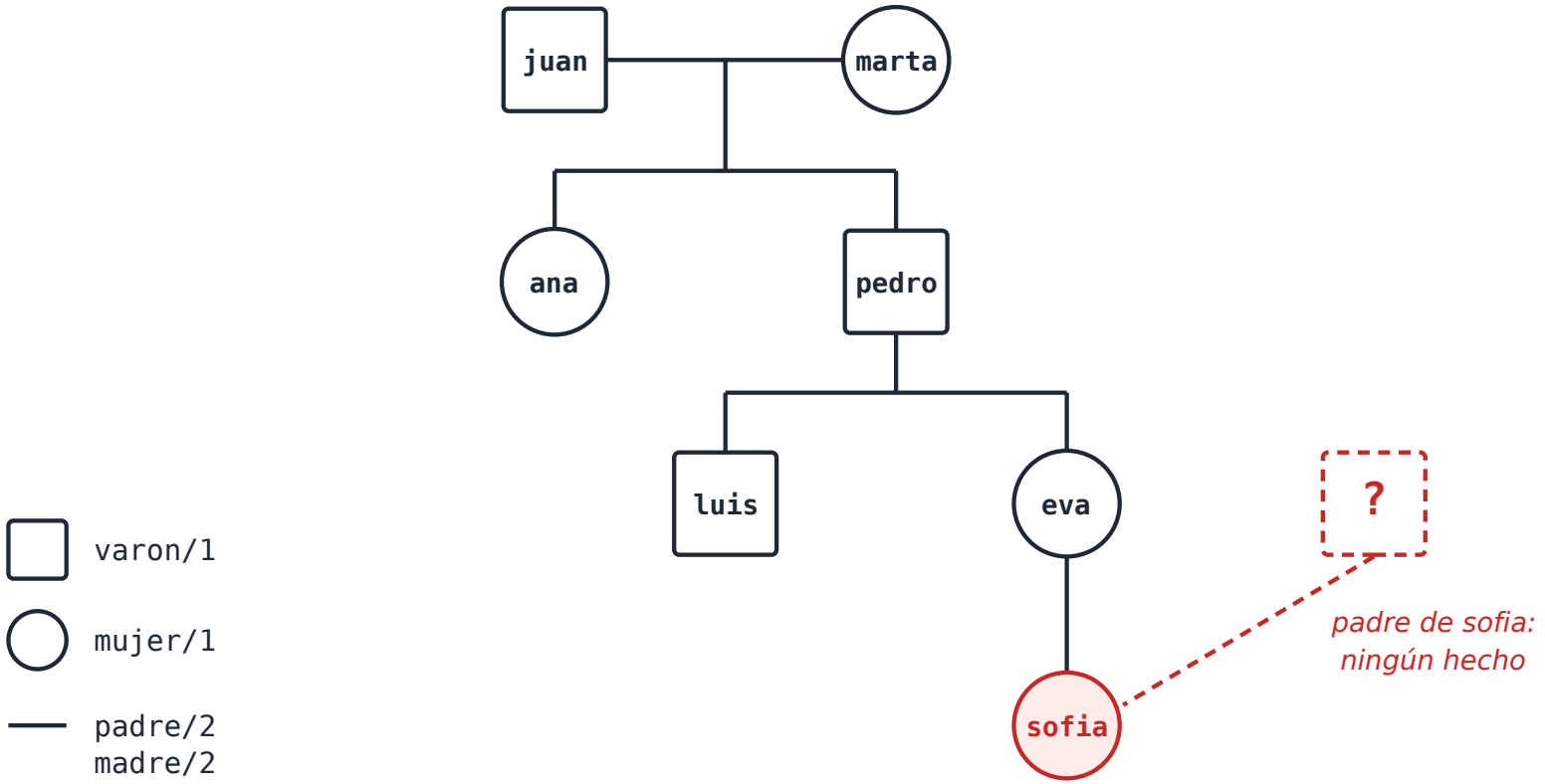
```
?- padre(juan, ana).
true.

?- padre(juan, luis).
false.

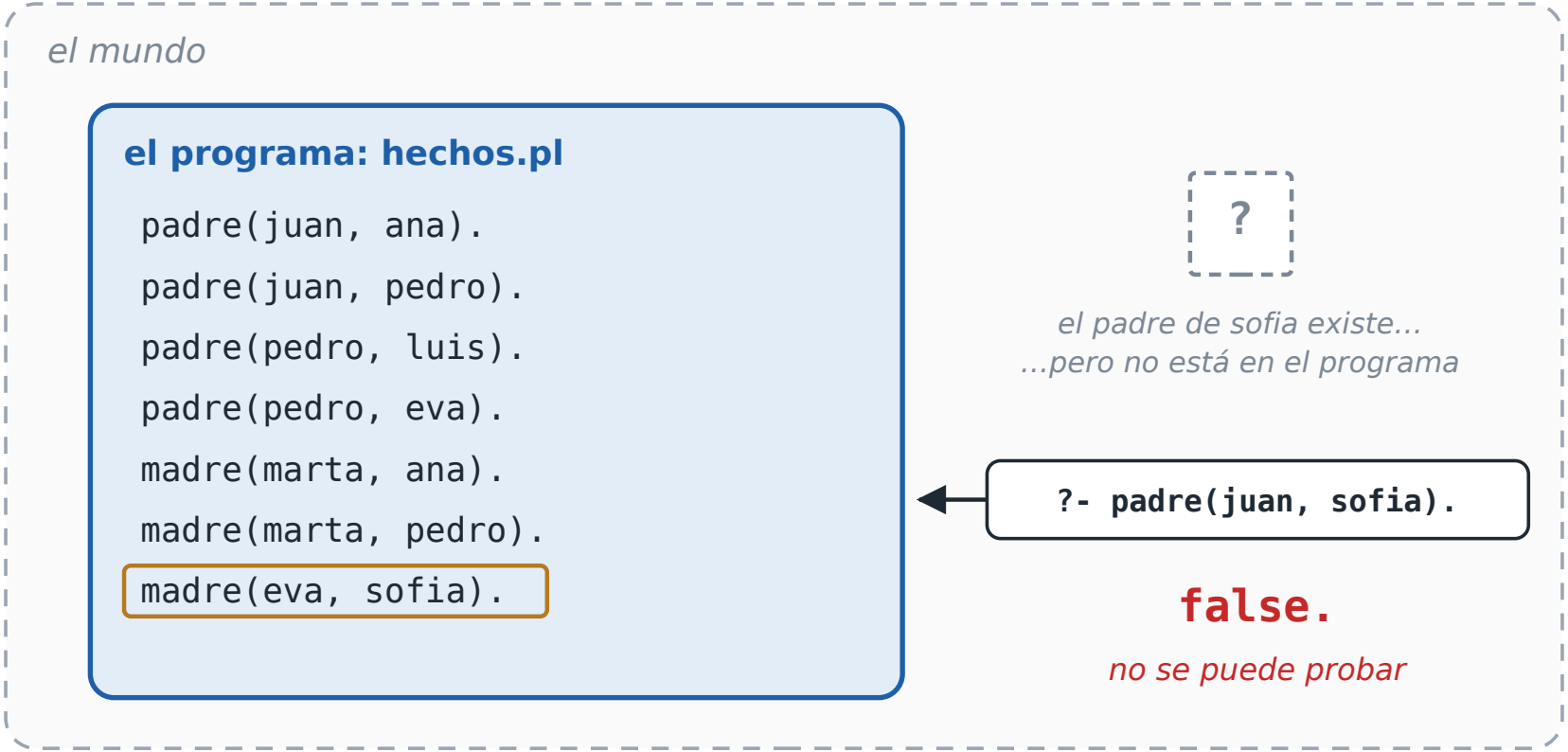
?- mujer(eva).
true.

?- padre(ana, juan).
false.
```

Lo que no está en el programa



El supuesto de mundo cerrado

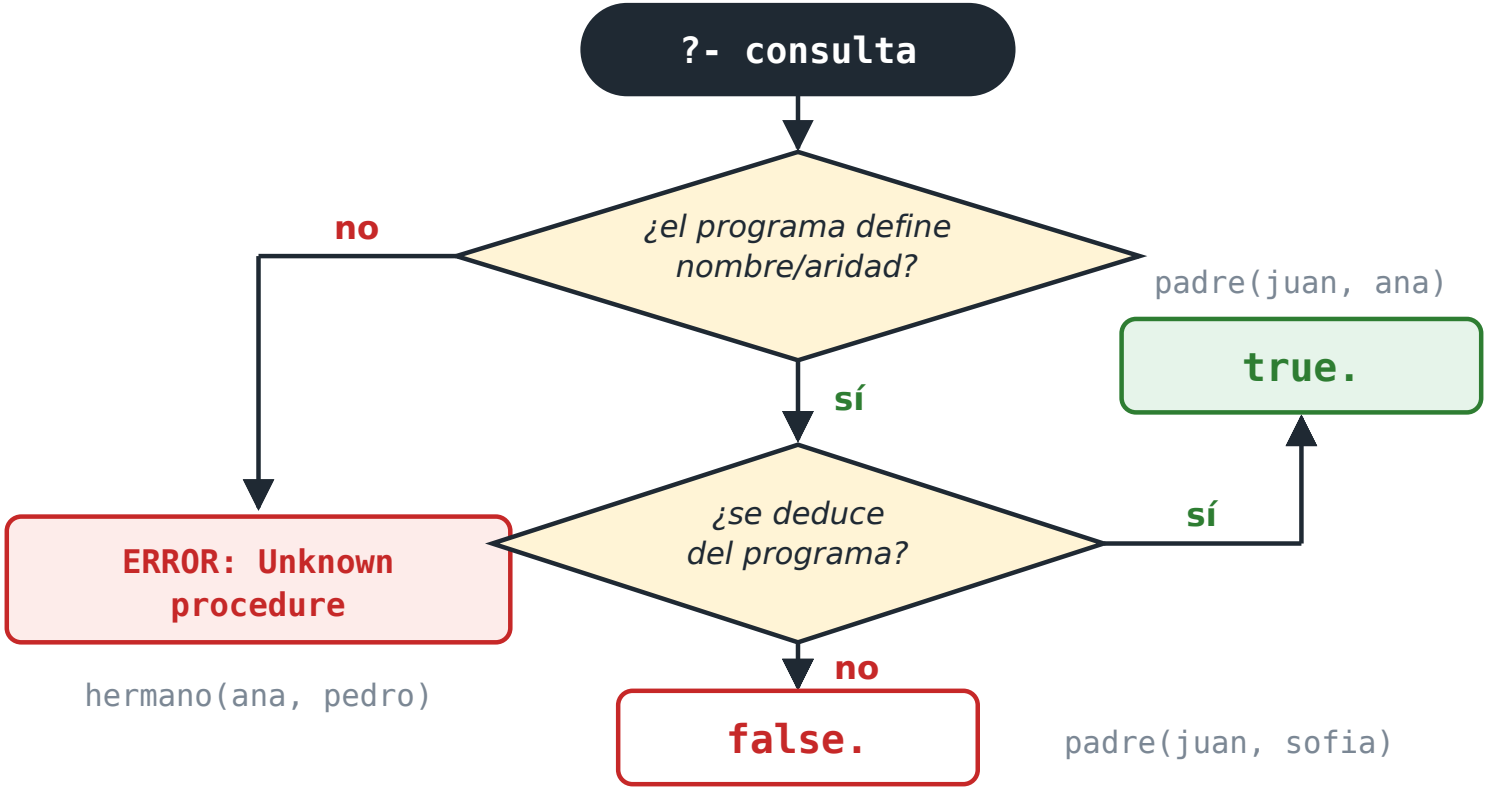


Dos resultados negativos

```
?- padre(juan, sofia).
false.

?- hermano(ana, pedro).
ERROR: Unknown procedure: hermano/2 (DWIM could not correct goal)
```

false. o ERROR



Un nombre parecido

```
?- padres(juan, ana).
Correct to: "padre(juan,ana)"?

ERROR: Unknown procedure: padres/2
ERROR:   However, there are definitions for:
ERROR:         padre/2
```

Los gustos de la familia

```
% gusta(P, C): a P le gusta C.
gusta(juan, futbol).
gusta(ana, prolog).
gusta(luis, futbol).
gusta(eva, prolog).
gusta(sofia, dibujar).
```

Preguntar quién o qué

```
?- gusta(ana, Que).
Que = prolog.

?- gusta(Quien, prolog).
Quien = ana ;
Quien = eva.

?- gusta(Quien, cocinar).
false.
```

Paso a paso (1)

```
?- gusta(Quien, prolog).
```

```
gusta(juan, futbol).
gusta(ana, prolog).
gusta(luis, futbol).
gusta(eva, prolog).
gusta(sofia, dibujar).
```

estados de Quien

libre

Quien *libre*

```
_
```

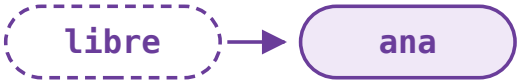
la consulta comienza:
Quien está libre

Paso a paso (2)

```
?- gusta(Quien, prolog).
```

```
gusta(juan, futbol).   ✗
gusta(ana, prolog).    ✓
gusta(luis, futbol).
gusta(eva, prolog).
gusta(sofia, dibujar).
```

estados de Quien



Quien **ana** *ligada*

```
Quien = ana ;
```

gusta(ana, prolog) coincide:
Quien queda ligada a ana

Paso a paso (3)

```
?- gusta(Quien, prolog).
```

```
gusta(juan, futbol).   ✗
gusta(ana, prolog).    ✓
gusta(luis, futbol).   ✗
gusta(eva, prolog).
gusta(sofia, dibujar).
```

estados de Quien



Quien *libre*

```
Quien = ana ;
```

el usuario ingresa ;
la ligadura se deshace

Paso a paso (4)

```
?- gusta(Quien, prolog).
```

```
gusta(juan, futbol).   ✗
gusta(ana, prolog).    ✓
gusta(luis, futbol).   ✗
gusta(eva, prolog).    ✓
gusta(sofia, dibujar).
```

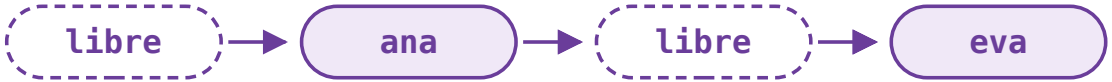
Quien **eva** *ligada*

```
Quien = ana ;
Quien = eva.
```

gusta(eva, prolog) coincide:
Quien queda ligada a eva

ningún otro hecho tiene prolog:
la respuesta termina con punto

estados de Quien



Dos incógnitas

```
?- gusta(Quien, Que).
Quien = juan,
Que = futbol ;
Quien = ana,
Que = prolog ;
Quien = luis,
Que = futbol ;
...
```

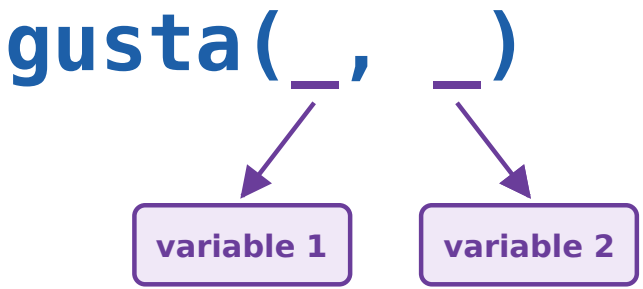
Una variable repetida



juan futbol	≠	ana prolog	≠	luis futbol	≠	eva prolog	≠	sofia dibujar	≠

false.

La variable anónima



independientes

```
?- gusta(_, dibujar).          true.
```

¿a alguien le gusta dibujar? — sin pedir a quién

Todas las respuestas



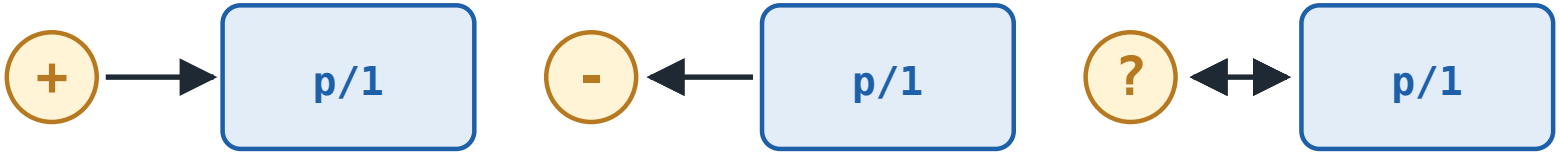

```
Quien = ana ;
```

termina en ; → puede haber más

```
Quien = eva.
```

termina en . → no queda ninguna

Cómo llega cada argumento



debe llegar ligado
entrada

el predicado lo liga
salida

libre o ligado

```
-Number is +Expr
```

is/2 en el manual de SWI-Prolog

is/2 en cada modo

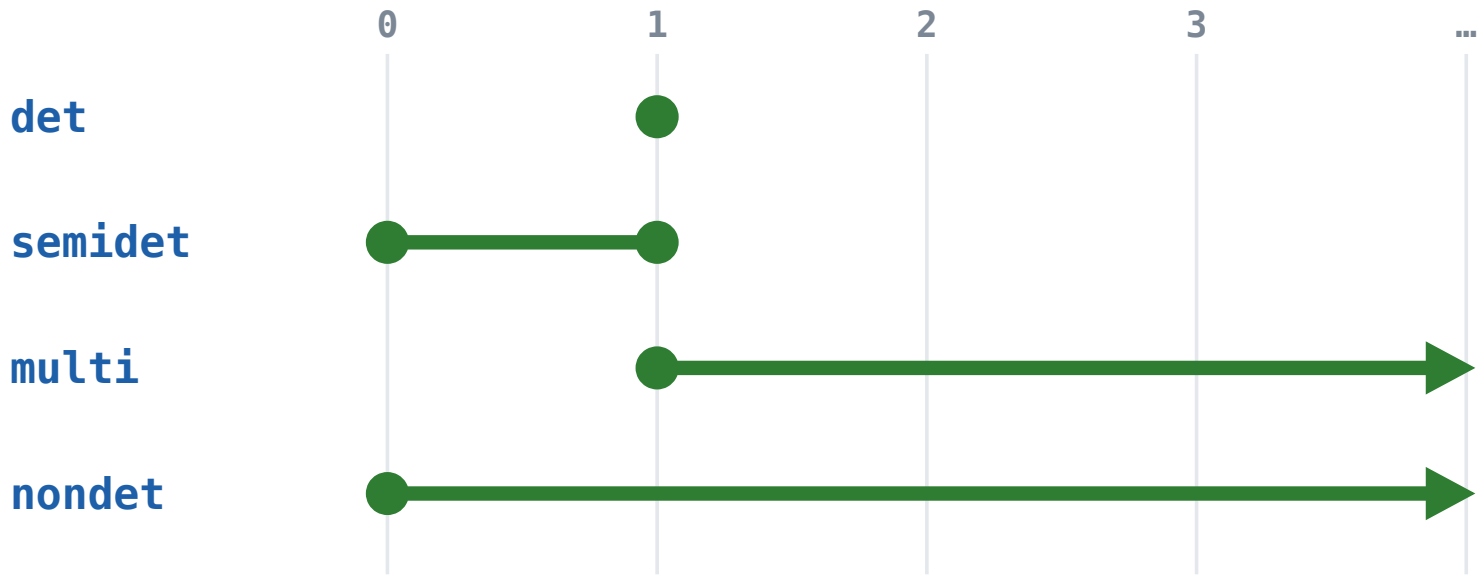
```
?- X is 2 + 3.
X = 5.

?- 5 is 2 + 3.
true.

?- X = 5, X is 2 + 3.
X = 5.

?- X is Y + 1.
ERROR: Arguments are not sufficiently instantiated
```

Cuántas respuestas



El encabezado de una regla

```
%!  abuelo(?A, ?N) is nondet.
%
%   A es abuelo de N cuando es el padre de su padre.
abuelo(A, N) :-
    padre(A, P),
    padre(P, N).
```